

A

B. Tech (Information Technology)

PROJECT REPORT ON

"DreamOS: An AI-Native Semantic File Management System with Knowledge Graph, Context Memory and Intelligent Workspace Management"

Submitted by

Name of student	University Exam No
Zaid Ali Syed	2305139
Om Vyas	2305170
Krushna Kadam	2305165

Under Guidance of

Dr. Minakshi R. Rajput

Institute of Information and Communication Technology (IICT)
MGM University, Chh. Sambhajinagar

Academic Year 2026-2027

Institute of Information and Communication Technology

MGM University, Chh. Sambhajinagar

CERTIFICATE

This is to certify that Zaid Ali Syed (Roll No. 2305139), Om Vyas (Roll No. 2305170) and Krushna Kadam (Roll No. 2305165) of B.Tech (IT), have successfully completed project work on "DreamOS: An AI-Native Semantic File Management System" under the guidance of Dr. Minakshi R. Rajput, and submitted the same during the academic year 2026-2027 towards the partial fulfillment of the degree of B.Tech (IT) from Institute of Information and Communication Technology, MGM University, Chh. Sambhajinagar.

Dr. Minakshi R. Rajput
Guide

Dr. S. C. Tamane
Director, IICT

Date: _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us in completing this project, DreamOS: An AI-Native Semantic File Management System.

We thank our project guide, Dr. Minakshi R. Rajput, for guidance, timely reviews and encouragement throughout the work. The reviews at each Project Monitoring checkpoint pushed us to test our claims, to report what the measurements did not show, and to keep the explanation of the system simple enough for any examiner to follow.

We thank Dr. S. C. Tamane, Director, IICT, and the faculty of the Institute of Information and Communication Technology, MGM University, for the academic environment, the laboratory facilities and the Project Monitoring structure in which this work was planned, reviewed and refined. We also thank the Research Methodology faculty for the requirement to prepare a research paper, which led us to design a controlled retrieval comparison and to record the defects we found along the way.

We are grateful to the open-source communities behind Ollama, ChromaDB, FastAPI, Tauri, React, NumPy and pytest, on whose work DreamOS is built, and to the researchers whose papers on semantic file systems and dense retrieval gave the project its direction.

Finally, we thank our families and friends for their patience and support during the long build, and each other for the shared work of design, implementation, testing and writing that this report records.

Zaid Ali Syed	**Om Vyas**	**Krushna Kadam**
Roll No. 2305139	Roll No. 2305170	Roll No. 2305165

CONTENTS

ACKNOWLEDGEMENT ii
ABSTRACT v
ABBREVIATIONS vi
LIST OF FIGURES vii
LIST OF TABLES viii
INTRODUCTION 1
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
LITERATURE REVIEW 3
2.1 Review of Existing Systems and Related Work 3
2.2 Research Gap 4
2.3 Summary of Findings 4
SYSTEM ANALYSIS 6
3.1 Requirement Analysis 6
3.1.1 Functional requirements 6
3.1.2 Non-functional requirements 7
3.2 Feasibility Study 8
3.3 System Specifications 8
SYSTEM DESIGN 10
4.1 System Architecture 10
4.2 Module Design 10
4.3 Data Flow Diagrams and UML Diagrams 11
4.3.1 Data flow diagrams 11
4.3.2 UML diagrams 12
4.4 Database Design 14
4.5 UI / UX Design 16
4.6 Technology Stack 17
IMPLEMENTATION 18
5.1 Module-wise Description 18
5.1.1 Natural Language Interface 18
5.1.2 Semantic Search Engine 18
5.1.3 AI File Organiser 18
5.1.4 Knowledge Graph 18

5.1.5 Context Memory Engine 18
5.1.6 Intelligent Workspace Manager 19
5.1.7 Privacy audit 19
5.2 Technologies Used 19
5.3 Code Snippets 19
5.4 Screenshots of Outputs 20
TESTING AND VALIDATION 22
6.1 Testing Types 22
6.2 Test Cases and Results 22
RESULTS AND DISCUSSION 24
7.1 Performance Evaluation 24
7.2 Comparison with Existing Systems 25
7.2.1 Capability comparison 25
7.2.2 Measured retrieval comparison 26
7.3 Observations 28
CONCLUSION AND FUTURE SCOPE 30
8.1 Summary of Work 30
8.2 Limitations 30
8.3 Future Enhancement 30
REFERENCES 32
APPENDICES 33
Appendix A Source code 33
Appendix B Scale benchmark raw results 33
Appendix C Research paper 33
Appendix D Papers and certificates 33

ABSTRACT

Personal computers hold thousands of files whose names were chosen once and rarely describe what the file is about now. Finding a file means remembering how it was named. DreamOS is an AI-native desktop file manager that lets a user describe a file in plain language, and it finds, opens, relates and organises files by meaning rather than by name. All processing runs on the user's computer: embeddings and language understanding come from a local model runtime, and the index is stored in local databases, so no file content or question is sent to a server.

The system has six modules: a natural-language interface that classifies intent, a semantic search engine, an AI file organiser with reversible moves, a knowledge graph of relationships between files, a context memory engine that resolves follow-up questions, and an intelligent workspace manager that recommends and creates groups of related files. A privacy audit reports every network destination the software uses, so the claim that data stays local can be checked rather than trusted.

Two evaluations were run. In a controlled retrieval comparison on 36 labelled queries, DreamOS placed the intended file first for 94% of queries, against 56% for a BM25 content index and 33% for filename matching; on the 10 queries that share no word with their target, both lexical baselines scored 0.00 and DreamOS scored 0.80. In a scale evaluation on synthetic vaults of 500, 2,000 and 5,000 files, indexing ran at 19.9 files per second at 500 files, search answered in a median of 34.5 ms, and the knowledge graph rebuilt in 3.2 seconds at 5,000 files; the original method took 246.1 seconds at only 2,000 files, and an intermediate version 143.3 seconds at 5,000. Defects found during measurement, a name-resolution delay on the test machine and three all-pairs computations in graph construction, were fixed and re-measured. The report states what the measurements do and do not show, including the small size of the labelled set and the synthetic nature of the scale data.

Keywords: semantic search, sentence embeddings, knowledge graph, local large language models, personal information management, privacy by design.

ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
BM25	Best Matching 25 (lexical ranking function)
CSS / HTML	Cascading Style Sheets / HyperText Markup Language
DFD	Data Flow Diagram
ER	Entity-Relationship
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
IICT	Institute of Information and Communication Technology
LLM	Large Language Model
MFT	Master File Table (NTFS file index)
MRR	Mean Reciprocal Rank
NLI	Natural Language Interface
SQLite	Structured Query Language database engine (file-based)
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

LIST OF FIGURES

Figure	Title	Page
4.1	System architecture: desktop client, local service, local model runtime and storage	10
4.2	Module design and the data each module passes to the next	11
4.3	Data flow diagram, context level	11
4.4	Data flow diagram, level 1	11
4.5	Use case diagram of user actions	12
4.6	Class diagram of response and settings classes	13
4.7	Sequence diagram of an open request	13
4.8	Sequence diagram of a follow-up question resolved from memory	14
4.9	Entity-relationship diagram	15
4.10	Chat view: related files with the reason each is linked	16
5.1	Knowledge-graph view with a selected file and its neighbours	20
5.2	Workspace view with a created workspace and suggestions	20
7.1	Scale results: graph rebuild time, indexing throughput and search latency	25
7.2	Hit@1 by system and query group, with 95% Wilson intervals	27

LIST OF TABLES

Table	Title	Page
2.1	Comparison with existing systems	3
3.1	Functional requirements	6
3.2	Non-functional requirements	7
3.3	Hardware and software specification	8
4.1	Fields of the files table	15
4.2	Tables added for the graph, memory and workspace modules	16
4.3	Technology stack by layer	17
6.1	Test levels and what each covered	22
6.2	Selected test cases and results	22
7.1	Scale benchmark - indexing and knowledge-graph build (after the fixes)	24
7.2	Search latency and retrieval quality on the synthetic vaults	24
7.3	Capability comparison with existing systems	25
7.4	Retrieval comparison on 36 labelled queries (37 files)	27
7.5	Effect of the defect fixes on the measured pipeline	28
B.1	Raw benchmark output at 500, 2,000 and 5,000 files	33

INTRODUCTION

1.1 Introduction

Digital work leaves a trail of documents: invoices, résumés, meeting notes, code, photographs of receipts and downloads with names such as final_final_v3.txt or Untitled document.txt. Operating systems organise these by folder and filename, both chosen at the moment of saving. As the number of files grows, the names stop matching the way people remember them. A person looks for “the résumé with the internship section” and must recall an arbitrary name instead.

DreamOS addresses this by making meaning the access key. The user types a request such as “find my invoices”, “open my resume”, “what is related to my resume” or “suggest some workspaces”, and the system responds by searching, opening, relating or grouping files. The application is a desktop shell built with Tauri and React that talks to a local Python service. The service stores its index in two local databases and uses a local language-model runtime for understanding. Nothing is sent to a remote server.

1.2 Problem Statement

Design and implement a locally run file-management layer that lets a person retrieve, open, relate and organise files by describing what they are about, without depending on exact filenames, folder locations or a cloud service, and that makes its privacy behaviour verifiable.

1.3 Objectives

- Build a semantic index of a file collection using local embedding models, so that files are found by meaning.
- Provide one natural-language interface that classifies what the user wants and routes the request to the right module.
- Return ranked results with honest confidence, and open a file only when the match is confident.
- Use a local language model to summarise, tag and categorise files, and apply that organisation reversibly.
- Model relationships between files, remember conversation so that follow-up questions work, and recommend workspaces.

- Keep all processing local, and provide an audit that reports where data could leave the machine.
- Measure the system against lexical baselines and at a scale beyond the demonstration vault, and report the results including the defects found.

1.4 Scope of the Project

The project covers a single-user desktop application for Windows, indexing a folder of text, Markdown, source code, PDF and Word documents. It includes the six modules described in the System Design chapter, a privacy audit, a controlled retrieval comparison and a scale evaluation on synthetic collections. It does not cover mobile devices, multi-user access, synchronisation across machines, cloud models, or indexing of images and audio. The demonstration uses a sandboxed vault of 37 generated files so that no real personal data is handled during evaluation.

LITERATURE REVIEW

2.1 Review of Existing Systems and Related Work

Two lines of research bear on this project: dense text embeddings, which place text in a vector space where distance reflects meaning, and semantic or AI-native file systems, which use such embeddings to organise files. Mikolov et al. showed that words can be embedded so that geometric distance tracks relatedness [2]. Reimers and Gurevych extended the idea to sentences and paragraphs with Sentence-BERT, which makes comparing whole documents efficient [3]. DreamOS uses nomic-embed-text, a model in this family, served locally through Ollama, with the Chroma vector database for nearest-neighbour search [4].

The closest system is LSFS, a semantic file system for an LLM operating-system agent presented at ICLR 2025 [1]. LSFS shows that embedding indexes plus a language-model front end can replace command-based file operations. It is designed for software agents that call an API, not for a person typing requests, and its evaluation does not establish that it runs entirely on local models. DreamOS takes the same architectural idea and exposes it to a human through one conversational surface that both finds and acts on files.

Mainstream desktop search is still driven by names and metadata. Windows Search indexes filenames, metadata and text but matches a query only where it shares vocabulary with the file [5]. Spotlight adds metadata and heuristics over a similar keyword index [6]. Everything indexes the NTFS master file table for instant filename lookup and does not read content [7].

Table 2.1 (page 3) compares the systems on their retrieval basis, whether content is indexed, the interface they offer, and whether they work locally.

Table 2.1 - Comparison with existing systems

System	Retrieval basis	Content indexed	Interface	Local operation	Limitation relative to DreamOS
Windows Search	Keyword and metadata index	Yes	Search box	Yes, local index	No meaning-based matching; cannot act on a result from one query

System	Retrieval basis	Content indexed	Interface	Local operation	Limitation relative to DreamOS
macOS Spotlight	Keyword, metadata, ranking heuristics	Yes	Search bar	Yes, local index	Not embedding-based; not conversational
Everything	NTFS file-name index (MFT)	No, names only	Search box	Yes, local index	Names only; no content understanding
LSFS (ICLR 2025)	LLM plus embedding index	Yes	Agent API	Not demonstrated	Built for agents via API; local operation not shown
DreamOS	Local embeddings plus local intent routing	Yes	One conversational interface	Yes, audited	Single-user desktop only; small-model limits on short queries

2.2 Research Gap

The literature establishes embedding-based file indexing as workable [1] and establishes that consumer tools match on words [5-7]. Three gaps remain. First, embedding search is presented as an agent API rather than as a single conversational surface that both retrieves and acts on files for a person. Second, published systems do not show that the full pipeline, from embedding to answer, runs on local models with the data-leaving question answered by measurement rather than assertion. Third, the behaviour of such systems on short, everyday queries, and on collections of thousands of files, is rarely reported with the defects that testing exposes.

2.3 Summary of Findings

- Embedding similarity is compressed for sentence models, so fixed similarity cut-offs drop correct answers to short queries; a relative cut-off with a small keyword boost recovers them without re-embedding.
- A language model's own judgement about whether a message refers back to earlier conversation is unreliable; deterministic rules and direct evidence should decide first.

- Pairwise similarity over a whole collection grows with the square of the number of files, and must be vectorised before it can scale.
- Averaging edge weights, rather than merging connected components, prevents unrelated groups from being joined by a single weak link.

SYSTEM ANALYSIS

3.1 Requirement Analysis

Requirements were derived from the problem statement and refined at each Project Monitoring review. Each requirement below has an identifier, a description of what it means in practice, and its status at submission. Functional requirements say what the system does; non-functional requirements say how well it must do it and how that is checked.

3.1.1 Functional requirements

Table 3.1 (page 6) lists the twelve functional requirements with a description of each.

Table 3.1 - Functional requirements

ID	Requirement	Description	Status
FR1	Accept one natural-language message through a chat interface	The user types a request in an ordinary sentence in one input box; no query syntax or file path is needed.	Implemented
FR2	Classify intent as search, open, related, workspace, organise or other	A local language model decides what the user wants so that the request reaches the right module instead of always searching.	Implemented
FR3	Semantic search returning ranked files with score and snippet	Files are ranked by meaning, each result shows a similarity score and a text snippet so that the match can be judged.	Implemented
FR4	Open the best match in its default application only when confident	A file is launched only above the confidence bar; otherwise candidates are shown, so a wrong file is never opened silently.	Implemented
FR5	Generate a summary, tags and category for each file	The model describes each file in one sentence and assigns tags and a category with the reason for it.	Implemented
FR6	Apply and reverse an organisation move	A file is moved into its category folder only on confirmation, and the original path is stored so the move can be undone.	Implemented
FR7	Preview unorganised files before any change	The user sees every proposed move before it is applied; nothing changes on disk during the preview.	Implemented

ID	Requirement	Description	Status
FR8	Index and re-index incrementally by content hash	Unchanged files are skipped on later runs, so re-indexing a large folder only costs time for what changed.	Implemented
FR9	Model relationships between files and show related files	Files are linked by similar content, direct reference and shared tags, and each link is shown with its reason.	Implemented
FR10	Resolve follow-up questions against the earlier conversation	Messages such as “open the second one” or “open it” are resolved using the files shown in earlier turns.	Implemented
FR11	Recommend and create workspaces from relationships and usage	Groups of related or frequently opened files are suggested, and a workspace opens all its files together.	Implemented
FR12	Report where data could leave the machine, from live configuration	A privacy view lists each network endpoint in use and whether it is on this computer.	Implemented

3.1.2 Non-functional requirements

Table 3.2 (page 7) lists the non-functional requirements, what each means for this system, and how it was evaluated.

Table 3.2 - Non-functional requirements

Attribute	Requirement	Description	How it is evaluated
Privacy	No file content or question leaves the machine	All models and stores run locally and the service listens on the loopback address only; remote endpoints would be flagged.	Privacy audit; netstat check; automated test
Performance	Search answers in well under a second	A user should not wait noticeably for a result once models are loaded.	Scale benchmark, Section 7.1
Scalability	Graph and indexing remain practical at several thousand files	Time to index and to rebuild relationships should grow gently with collection size.	Scale benchmark at 500, 2,000 and 5,000 files

Attribute	Requirement	Description	How it is evaluated
Reliability	Organisation moves are reversible	No action may lose or overwrite an original file; every move can be undone.	Automated apply-and-revert test
Usability	One input box; results are cards with actions	A new user can ask in plain language and act on a result with one click.	Screenshots, Section 4.5
Maintainability	Each module has its own tests; offline test run	Changes can be checked quickly without a model runtime.	123 automated tests, CI on every push

3.2 Feasibility Study

Technical feasibility was confirmed early by building the indexing and search path with a local embedding model and measuring it on the demonstration vault. Economic feasibility holds because every component is open source and the models run on an ordinary laptop without a graphics card, so the running cost is zero. Operational feasibility depends on the user accepting that the first request after a cold start is slow while a model loads, which is documented in the demonstration guide. Schedule feasibility was managed by building the three core modules first and then adding the graph, memory and workspace modules as separate, tested increments.

3.3 System Specifications

Table 3.3 (page 8) gives the hardware and software used for development and for every measurement in this report.

Table 3.3 - Hardware and software specification

Item	Specification used for development and evaluation
Processor	AMD Ryzen 7 6800H, 8 cores, 16 logical processors
Memory	16 GB installed (about 15 GB reported as usable)
Storage	Local solid-state storage; index size is reported per run in the benchmark output
Graphics	Not required; models run on the CPU
Operating system	Windows 11
Language and frameworks	Python 3.12, FastAPI, Pydantic, ChromaDB 1.5, SQLite 3, NumPy 2.5
Models	nomic-embed-text (137M parameters, embeddings); llama3.2 3B (intent, summaries, tags)

Item	Specification used for development and evaluation
Desktop shell	Tauri 2 with React 19, TypeScript and Vite
Testing	pytest 9, FastAPI test client, GitHub Actions on Linux
Packaging	Windows MSI and NSIS installers built by the Tauri bundler

SYSTEM DESIGN

4.1 System Architecture

DreamOS is a client-server application on one machine. The desktop client calls a local service over HTTP on 127.0.0.1:8420. The service classifies requests, searches, organises and computes relationships, and calls the local model runtime for language tasks. Two stores hold the data: a relational database for metadata, relationships, conversation and usage, and a vector database for the embeddings of text chunks.

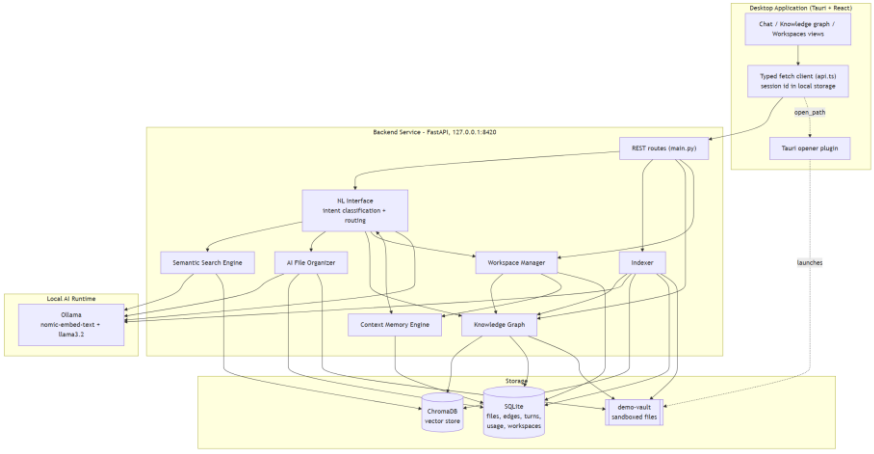


Figure 4.1 - System architecture: desktop client, local service, local model runtime and storage

Figure 4.1 shows the four zones of the system. The desktop application (top right) holds the React views, a typed fetch client that keeps the session identifier, and the Tauri opener plugin that launches files. The backend service (centre) receives every request at its REST routes and hands chat messages to the Natural Language Interface, which calls the search engine, organiser, workspace manager, knowledge graph or context memory as the intent requires; the indexer is started from its own route. The local AI runtime (left) is called for embeddings and completions. Storage (bottom) holds vectors in ChromaDB, relational data in SQLite and the sandboxed vault files. The dotted line shows that a file is launched by the Tauri opener on the client, not by the service.

4.2 Module Design

The system is organised in six modules. The Natural Language Interface decides what a request means and routes it. The Semantic Search Engine returns files near the best match. The AI File Organiser proposes categories and reversible moves. The Knowledge Graph links files by similar content, by direct reference and by shared tags. The Context Memory Engine keeps the conversation so that a follow-up can be resolved, and records which files the user opens. The

Workspace Manager turns the graph and usage into explainable recommendations and creates workspaces that open together.

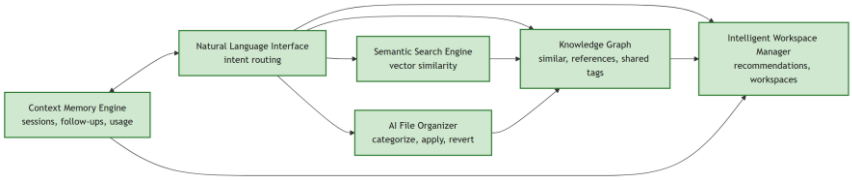


Figure 4.2 - Module design and the data each module passes to the next

Figure 4.2 shows how control and data pass between the six modules. The Natural Language Interface is at the centre of control: it exchanges turns with the Context Memory Engine and dispatches requests to the Semantic Search Engine, the AI File Organizer, the Knowledge Graph and the Intelligent Workspace Manager. Search results and organiser output feed the Knowledge Graph, the Knowledge Graph feeds the Workspace Manager, and the Context Memory Engine supplies usage data to the Workspace Manager as well.

4.3 Data Flow Diagrams and UML Diagrams

4.3.1 Data flow diagrams

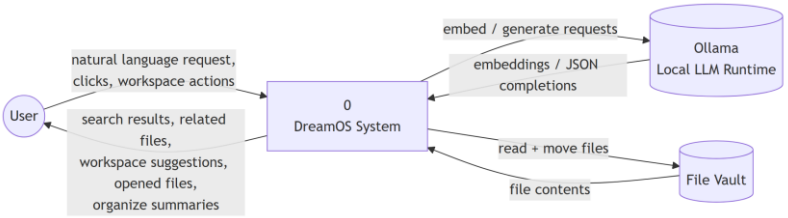


Figure 4.3 - Data flow diagram, context level

The context-level diagram treats DreamOS as a single process. The user sends natural-language requests, clicks and workspace actions, and receives search results, related files, workspace suggestions, opened files and organisation summaries. The process exchanges embedding and generation requests with the local Ollama runtime and reads from, and moves files in, the file vault. No other external entity appears, which is the visual form of the claim that the system is self-contained.

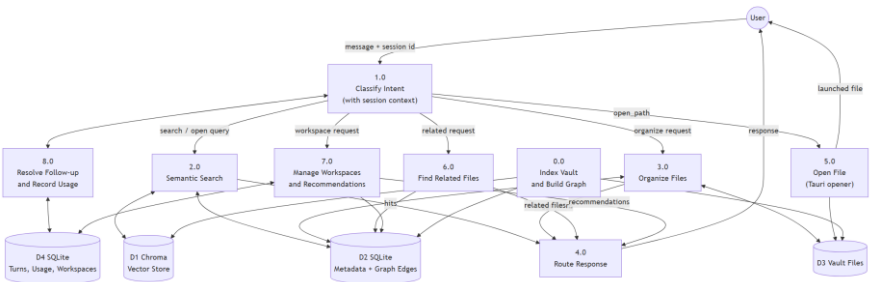


Figure 4.4 - Data flow diagram, level 1

Level 1 splits the system into processes: 0.0 indexes the vault and builds the graph; 1.0 classifies intent with session context; 2.0 runs semantic search; 3.0 organises files; 4.0 routes the response; 5.0 opens a file through the Tauri opener; 6.0 finds related files; 7.0 manages workspaces and recommendations; and 8.0 resolves follow-ups and records usage. Four data stores are shown: the Chroma vector store (D1), SQLite metadata and graph edges (D2), the vault files (D3), and SQLite turns, usage and workspaces (D4).

4.3.2 UML diagrams

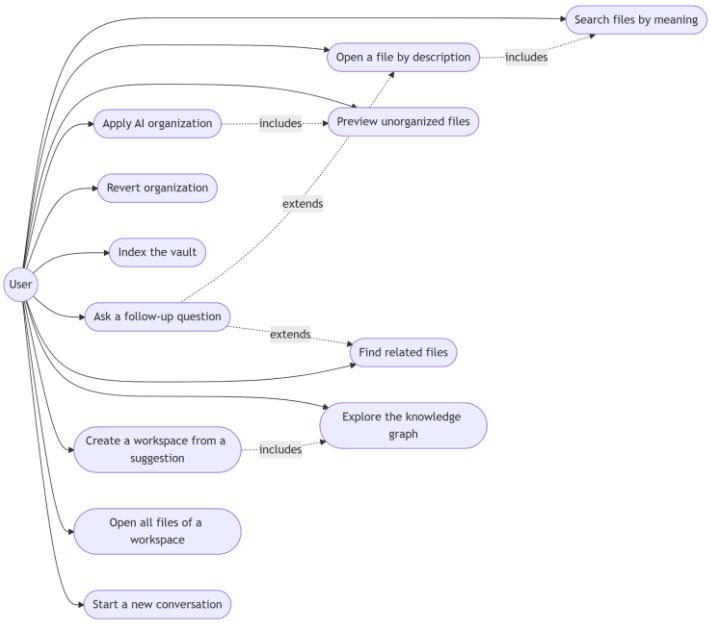


Figure 4.5 - Use case diagram of user actions

The use case diagram shows what the single actor, the user, can do: search files by meaning; open a file by description (which includes a search); preview unorganised files and apply or revert AI organisation (applying includes the preview); index the vault; ask a follow-up question (which extends opening a file and finding related files); find related files; explore the knowledge graph; create a workspace from a suggestion (which includes exploring the graph); open all files of a workspace; and start a new conversation.

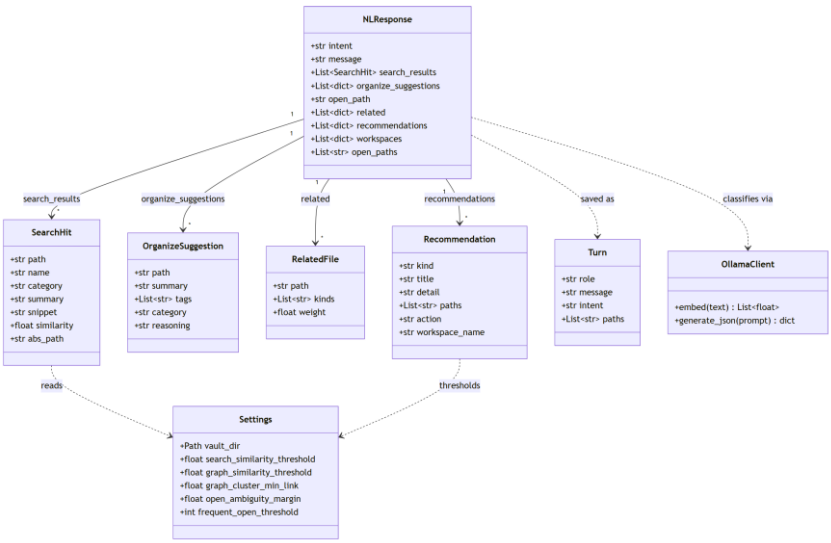


Figure 4.6 - Class diagram of response and settings classes

The class diagram shows the data classes exchanged between modules. NLResponse is the single object returned for every message; it carries the intent, a message and, depending on the intent, search results, organisation suggestions, an open path, related files, recommendations and workspaces. SearchHit, OrganizeSuggestion, RelatedFile and Recommendation are its elements, Turn is how a message is saved, OllamaClient offers embed and generate_json, and Settings holds the thresholds that scoring and the recommendation rules read.

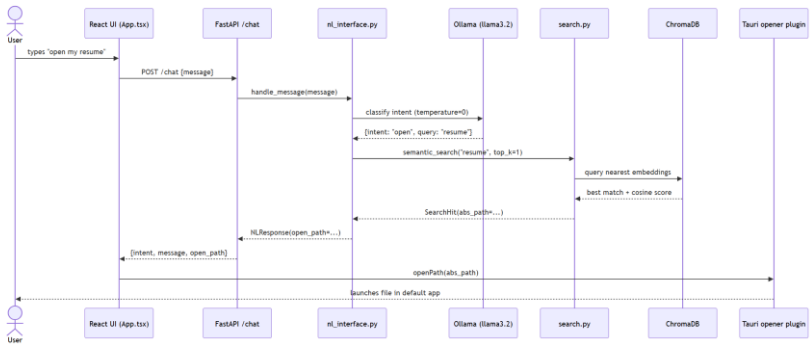


Figure 4.7 - Sequence diagram of an open request

The sequence follows the request “open my resume”. The React interface posts the message to /chat; the interface module asks llama3.2 to classify it at temperature 0, which returns intent open and query resume. Semantic search then asks ChromaDB for the nearest embeddings and returns the best match with its cosine score. The response carries the open path back to the interface, which asks the Tauri opener plugin to launch the file in its default application.

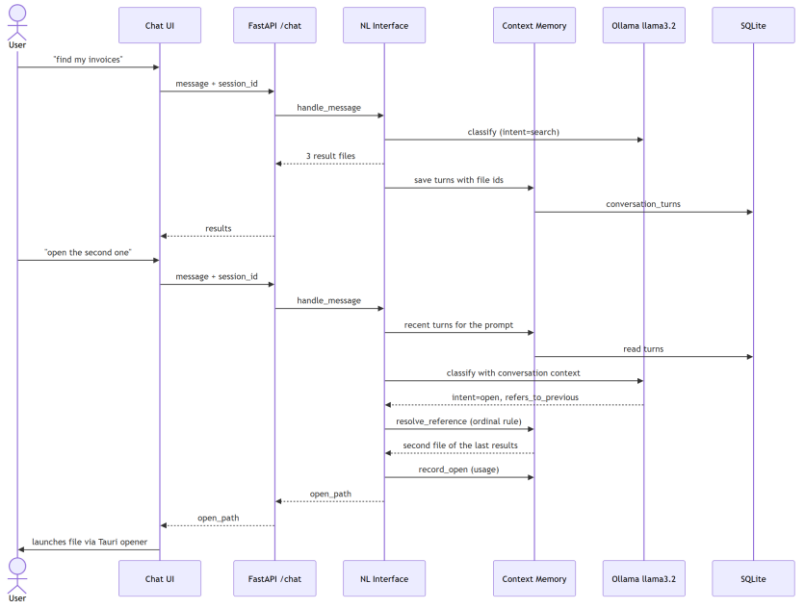


Figure 4.8 - Sequence diagram of a follow-up question resolved from memory

Two turns are shown. In the first, “find my invoices” is classified as a search, three files are returned and the turn is saved with their identifiers. In the second, “open the second one” is classified with the recent turns as context; the model returns intent open with a refers-to-previous flag, the ordinal rule resolves the second file of the last results from the Context Memory Engine, the open is recorded as a usage event, and the path is returned to the client for launching.

4.4 Database Design

File metadata is stored in SQLite and each file's text chunks are stored as vectors in ChromaDB, linked by the file path. All relationship, memory, usage and workspace tables refer to the file's numeric identifier rather than its path. The organiser renames a file when it moves it, but the identifier does not change, so no table needs rewriting after a move.

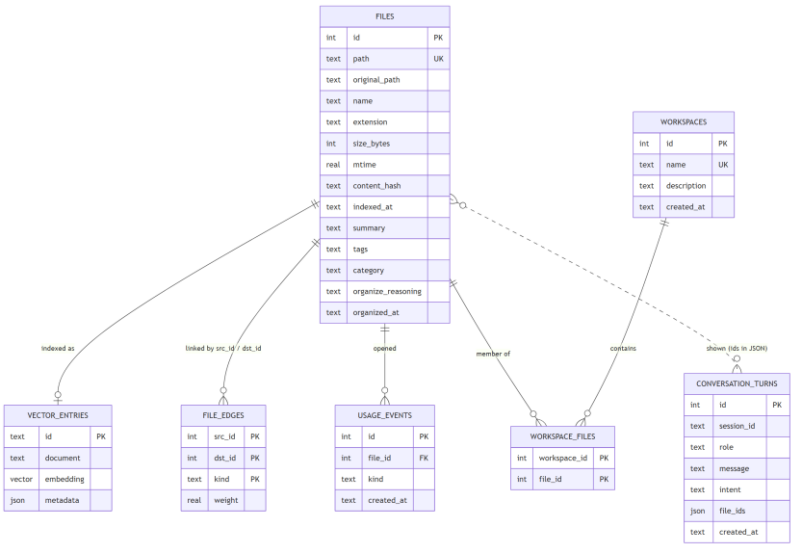


Figure 4.9 - Entity-relationship diagram

The entity-relationship diagram centres on the FILES table, whose integer id is referenced by every other table. VECTOR_ENTRIES holds the chunk vectors, linked to a file through its path. FILE_EDGES links two files by src_id and dst_id with a kind and a weight. USAGE_EVENTS records each open. WORKSPACES and WORKSPACE_FILES give named groups of files, and CONVERSATION_TURNS stores each turn with the identifiers of the files it showed, kept as a JSON list.

Table 4.1 (page 15) gives the fields of the central files table.

Table 4.1 - Fields of the files table

Field	Type	Purpose
id	INTEGER	Stable identifier used by every other table
path	TEXT, unique	Path relative to the vault; also the vector identifier
original_path	TEXT	Path at first indexing, kept so a move can be reversed
content_hash	TEXT	Lets re-indexing skip unchanged files
summary, tags, category	TEXT	Produced by the organiser
organize_reasoning, organized_at	TEXT	Explanation and time of the applied organisation

Table 4.2 (page 16) lists the tables added for the graph, memory and workspace modules.

Table 4.2 - Tables added for the graph, memory and workspace modules

Table	Key fields	Purpose
file_edges	source, target, kind, weight	Typed relationships between files
conversation_turns	session, role, message, file identifiers	Conversation kept across restarts
usage_events	file identifier, kind, time	Records each open; used for tie-breaking and recommendations
workspaces, workspace_files	name; workspace and file identifiers	Named groups of files

4.5 UI / UX Design

The interface has four views: a chat view for requests, a knowledge-graph view, a workspace view and a privacy view. Results appear as cards with the action the user can take, so a file is opened from the card that shows it. Related-file answers show why each file is linked, such as similar content, a direct reference or shared tags.

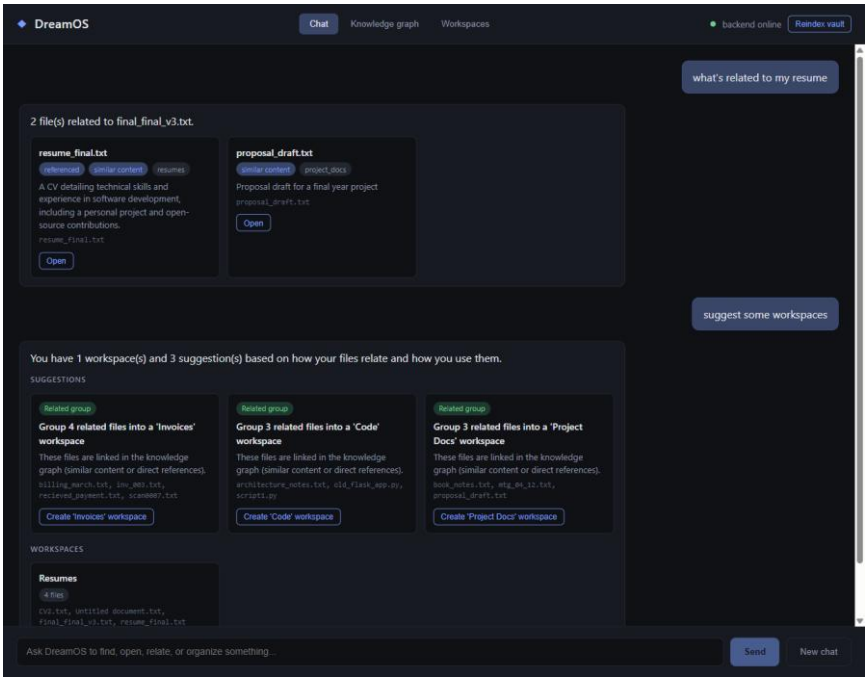


Figure 4.10 - Chat view: related files with the reason each is linked

The chat view shows the answer to “what's related to my resume”, with a reason chip on each result card (referenced, similar content), followed by the answer to “suggest some workspaces”,

which offers three suggested groups and lists the existing workspace. Every card carries the action available for it, such as Open or Create workspace.

4.6 Technology Stack

Table 4.3 (page 17) lists the technology used in each layer.

Table 4.3 - Technology stack by layer

Layer	Technology
Interface	React 19, TypeScript, Vite, d3-force for the graph layout, Tauri 2 shell
Service	Python 3.12, FastAPI, Uvicorn, Pydantic
Understanding	Ollama runtime with nomic-embed-text and llama3.2
Storage	SQLite for relational data; ChromaDB for vectors
Numerics	NumPy for vectorised similarity
Quality	pytest, GitHub Actions, Playwright and the WebView2 debug protocol for end-to-end checks

IMPLEMENTATION

5.1 Module-wise Description

5.1.1 Natural Language Interface

Each message is sent to the local model with a short description of the intents, and the model returns JSON with the intent, the topic to search for, and a flag for whether the message refers back to earlier conversation. Requests are routed deterministically after that. The interface reports a low-confidence result as a guess and never launches a file on a weak match.

5.1.2 Semantic Search Engine

A query is embedded and compared with every chunk. Results are grouped by file. A file is kept if it lies within a margin of the best hit and above a floor, and a small boost is added where the query's words appear in the file's text or name. This replaced a fixed cut-off, which had returned nothing for short queries such as “meeting”.

5.1.3 AI File Organiser

For each file the model proposes a summary, tags, a category and the reason for the category. The user applies or refuses each proposal. Applying moves the file into a folder named after the category and records the original path. Reverting moves it back and removes the folder if it is empty.

5.1.4 Knowledge Graph

Three kinds of edge are computed: similar content, from the cosine similarity of the centred mean embedding of each file; direct reference, when one file's text names another file; and shared tags, when two files share at least two tags. Each file keeps its three strongest neighbours. Clusters are built by average-linkage merging, which joins two groups only while the average weight between their members stays above a threshold. Two steps were made fast enough for thousands of files: the reference check files each file name under its rarest alphanumeric token so that a text is searched only for the names whose token it contains, and the clustering keeps its candidate merges in a heap over per-group adjacency instead of rescanning every link after each merge.

5.1.5 Context Memory Engine

Every turn is stored with the identifiers of the files it showed. A follow-up such as “open the second one” or “open it” resolves to a file by rule. Only when no rule applies, and no confident

search matches the request, is the model's own judgement used. This ordering was introduced after a live failure described in Section 7.3.

5.1.6 Intelligent Workspace Manager

Recommendations come from rules, not from the language model, so they are reproducible and explainable. They include related groups, frequently opened files, possible duplicates, files without a category and stale files. A workspace is a named group whose files open together.

5.1.7 Privacy audit

The audit reads the live configuration and lists each network endpoint the service uses, whether it is on this computer, what it is used for, and what is stored locally. It says plainly that it checks configuration, not network traffic, and shows how to confirm the listening address with netstat.

5.2 Technologies Used

The service is written in Python with FastAPI, which provides typed request models and an automatic test client. ChromaDB stores vectors on disk, and SQLite stores everything relational. NumPy performs the pairwise similarity computation as a single matrix product. The desktop shell is Tauri, which hosts a React interface and runs the operating-system file opener under an explicit permission scope. The language model runtime is Ollama, which serves both models over a local HTTP interface.

5.3 Code Snippets

The following excerpt is the vectorised similarity computation. One matrix product gives every pairwise cosine similarity, and the strongest neighbours of each file are selected in one pass. The original version compared each pair in a Python loop.

```
unit = matrix / norms[:, None]
sims = unit @ unit.T
np.fill_diagonal(sims, -np.inf)
k = min(settings.graph_top_k, len(paths) - 1)
neighbours = np.argpartition(-sims, k - 1, axis=1)[:, :k]
```

The excerpt below is the rule that decides whether a follow-up refers to earlier results. Deterministic evidence is checked first, and the model's flag is only a last resort.

```
def _open(query, referenced, weak_referenced=None):
    if referenced:                          # rule-based: ordinal or bare "it"
        return _open_response(search.hit_for_path(referenced))
    hits = search.semantic_search(query, top_k=3)
    best = hits[0] if hits else None
    if not best or best.similarity < settings.search_similarity_threshold:
        if weak_referenced:                 # model's guess, last resort only
            return _open_response(search.hit_for_path(weak_referenced))
```

5.4 Screenshots of Outputs

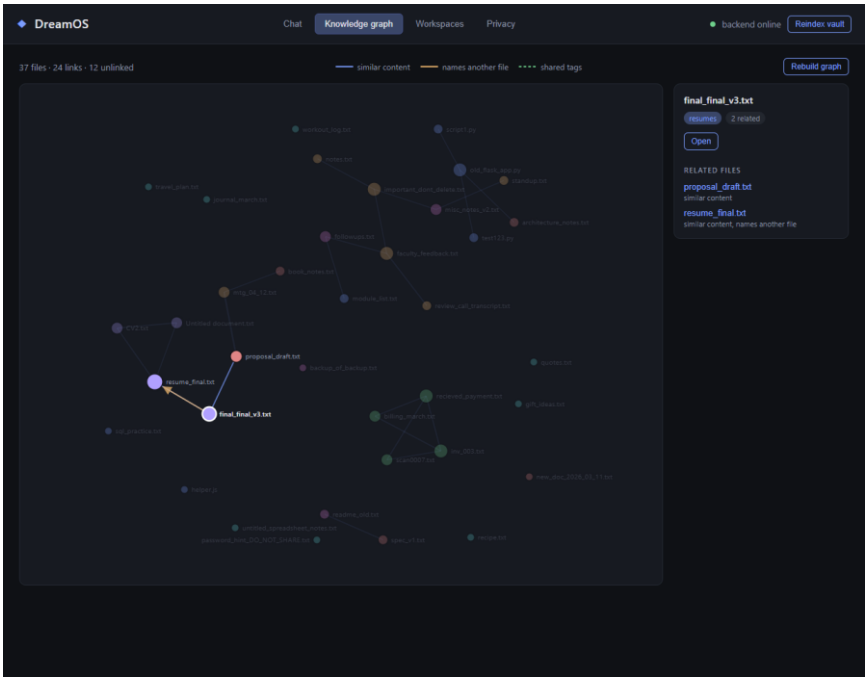


Figure 5.1 - Knowledge-graph view with a selected file and its neighbours

The graph view lays out the 37 files with 24 links; 12 files have no link. Edge colour shows the kind of relationship (similar content, names another file, shared tags). Selecting final_final_v3.txt highlights its neighbours and lists them in the side panel with the reason for each link, here similar content with proposal_draft.txt and similar content plus a direct reference to resume_final.txt.

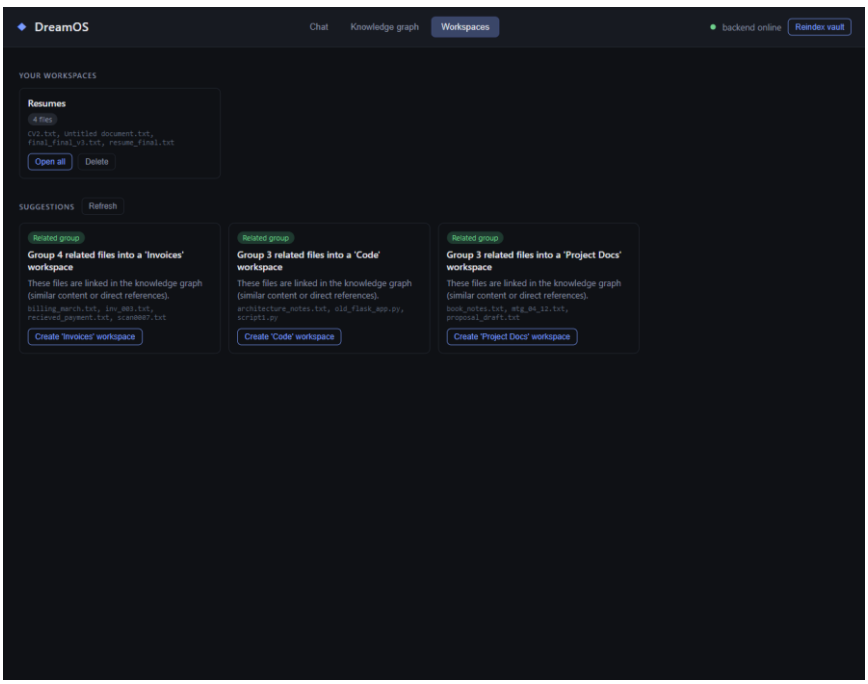


Figure 5.2 - Workspace view with a created workspace and suggestions

The workspace view lists the workspaces already created, each with Open all and Delete actions, above the rule-based suggestions. Each suggestion names the group, explains why the files belong together and lists them, with a single button that creates the workspace.

TESTING AND VALIDATION

6.1 Testing Types

Table 6.1 (page 22) summarises the test levels and what each covered.

Table 6.1 - Test levels and what each covered

Level	What was tested	Where
Unit	Cosine and centring, cut-off and boost, reference detection, cluster merging, ordinals, memory rules	tests/test_*.py
Integration	Indexing to search, organise and revert across the database and vector store; intent routing with the memory	tests/test_nl_interface.py, test_indexer.py
API	Every HTTP route through the FastAPI test client, including error codes	tests/test_api.py
Equivalence	The vectorised similarity, the token-indexed reference check and the heap clustering against their original all-pairs definitions on random data	tests/test_knowledge_graph.py
System	Live runs against the real model runtime and vault; desktop shell driven over the WebView2 debug protocol	Sections 6.2 and 7.1

6.2 Test Cases and Results

The automated suite runs offline: model calls are replaced by fixtures that return fixed vectors, so the suite needs no model runtime and runs in well under a minute. It runs on every push in continuous integration, and all 123 tests passed at the time of writing. The table shows selected cases; each one was also run against the live models.

Table 6.2 (page 22) lists the selected test cases with the expected and observed results.

Table 6.2 - Selected test cases and results

Test case	Expected	Result
Find a file by meaning, not name	Relevant file ranked first	Passed; “invoice from the hosting vendor” returns the vendor invoice
Short query with a weak but correct best hit	Returned, labelled as low confidence	Passed

Test case	Expected	Result
Open a weak match	Not launched; candidates shown	Passed
Open “the second one” after a search	Second result opened	Passed
Follow-up on a new topic after an old one	New topic wins over stale context	Passed after fix (Section 7.3)
Apply then revert an organisation	File returns to original place; empty folder removed	Passed
Relationships survive a file move	Edges still attached to the moved file	Passed
Vectorised similarity equals original loop	Identical edges on random data	Passed
Remote model endpoint reported	Marked as leaving the machine	Passed
Workspace create then open all	Every file opened	Passed in the desktop shell

RESULTS AND DISCUSSION

7.1 Performance Evaluation

Scale was measured on synthetic collections of 500, 2,000 and 5,000 files. The collections were generated across twelve topics, such as invoices, recipes and research notes, and about half of the documents include one sentence from an unrelated topic, so that the collection is not trivially separable. Each run used a fresh database and measured the real pipeline with the real models: embedding through Ollama, storage in ChromaDB and SQLite, graph construction, and 36 search queries with their latency. The benchmark script is part of the repository (backend/scripts/bench_scale.py) so that the measurements can be reproduced.

Table 7.1 (page 24) reports indexing and knowledge-graph build after the fixes.

Table 7.1 - Scale benchmark - indexing and knowledge-graph build (after the fixes)

Files	Index time (s)	Index rate (files/s)	Graph rebuild (s)	Graph links
500	25.2	19.9	0.4	1062
2,000	86.1	23.2	1.3	4747
5,000	276.3	18.1	3.2	11938

Index time includes the knowledge-graph rebuild, which runs after each indexing pass that changed something. Before the vectorisation the same measurement at 2,000 files was 372.5 seconds for indexing, including a graph rebuild of 246.1 seconds. After vectorising the similarity step the 5,000-file rebuild was still 143.3 seconds, and profiling showed that 98% of it was the reference check, an all-pairs regular-expression search. Filing each file name under its rarest alphanumeric token, so that a text is only searched for names whose token it contains, and replacing the clustering rescan with a heap over per-group adjacency, brought the 5,000-file rebuild to 3.2 seconds with the same 11938 links. At that size the graph is 1% of the 276.3 second index pass; embedding the files now dominates.

Table 7.2 (page 24) reports search latency and the retrieval-quality sanity check on the synthetic data.

Table 7.2 - Search latency and retrieval quality on the synthetic vaults

Files	p50 latency (ms)	p95 latency (ms)	Precision@5	Queries with no result
500	34.5	60.7	1.00	0

Files	p50 latency (ms)	p95 latency (ms)	Precision@5	Queries with no result
2,000	106.5	126.7	1.00	0
5,000	204.7	232.7	1.00	0

Precision@5 is the share of the five best results that come from the topic the query describes. On this synthetic data it stays at the maximum, so it shows that the topics separate, but it does not measure retrieval on real documents. The controlled comparison in Section 7.2 addresses that on the demonstration vault.

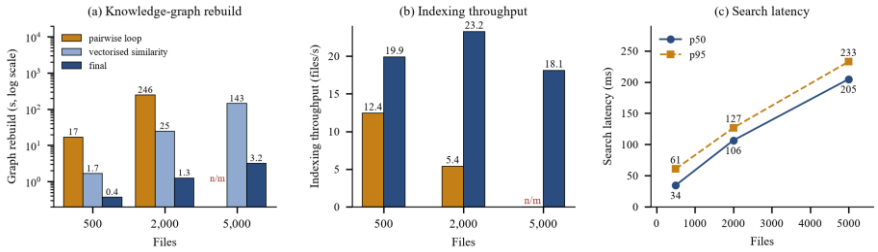


Figure 7.1 - Scale results: graph rebuild time, indexing throughput and search latency

Figure 7.1 plots the same runs as Tables 7.1 and 7.2. Panel (a) shows the knowledge-graph rebuild time on a logarithmic scale, with the original pairwise loop, the vectorised-similarity version and the final version side by side; the 5,000-file loop was not measured. Panel (b) shows indexing throughput, which includes the graph rebuild. Panel (c) shows median and 95th-percentile search latency, which grows with the number of stored vectors.

7.2 Comparison with Existing Systems

7.2.1 Capability comparison

Table 7.3 (page 25) compares the capabilities of DreamOS with the systems reviewed in the Literature Review chapter. The entries for the other systems follow their published documentation; vendor features added later were not evaluated, and LSFS was assessed from its paper only.

Table 7.3 - Capability comparison with existing systems

Capability	Windows Search	Spotlight	Everything	LSFS	DreamOS
Matches by meaning (embeddings)	No	No	No	Yes	Yes

Capability	Windows Search	Spotlight	Everything	LSFS	DreamOS
Indexes file content	Yes	Yes	No (names)	Yes	Yes
Conversational requests with follow-ups	No	No	No	No (agent API)	Yes
Opens a file only above a confidence bar	No	No	No	Not assessed	Yes
Acts on results (organise, reverse)	Open only	Open only	Open only	Yes, for agents	Yes, with confirmation
Typed relationship graph with reasons	No	No	No	Not assessed	Yes
Runs entirely on local models	Not applicable	Not applicable	Not applicable	Not shown	Yes; checked by audit
Verifiable privacy report	No	No	No	No	Yes (configuration level)

7.2.2 Measured retrieval comparison

The capability table is qualitative, so a measured comparison was added. The corpus is the 37-file demonstration vault. 36 queries were written, and the relevant file or files for each were fixed before any system was run. Each query is then partitioned automatically by whether any of its content words, after the system's own stop-word and plural folding, occurs in a relevant file's name or text: 26 queries do and 10 do not. Four systems return at most 8 files: filename match (a lexical match on names only, standing in for name-index tools); BM25 over name and text (standing in for lexical content indexes); DreamOS embeddings only (without the keyword boost and cut-off); and DreamOS as shipped. The metrics are Hit@1, Hit@3 and mean reciprocal rank (MRR). The script is backend/scripts/eval_retrieval.py and the committed output is docs/benchmarks/retrieval_comparison.json.

Table 7.4 (page 27) gives the results.

Table 7.4 - Retrieval comparison on 36 labelled queries (37 files)

System	Hit@1 all	Hit@3 all	MRR all	Hit@1 shares words (n=26)	Hit@1 no shared words (n=10)	MRR no shared words
Filename match	0.33	0.36	0.35	0.46	0.00	0.00
BM25 over name and text	0.56	0.67	0.61	0.77	0.00	0.00
Embedding only	0.92	1.00	0.95	0.92	0.90	0.95
DreamOS (shipped)	0.94	1.00	0.97	1.00	0.80	0.90

DreamOS places a relevant file first for 94% of all queries, against 56% for BM25 and 33% for filename match. The partition explains the gap. When a query shares a word with its target, BM25 reaches Hit@1 0.77 and DreamOS 1.00. When it shares none, both lexical baselines score exactly 0.00 by construction, while DreamOS scores 0.80 with every target inside its top three. Examples are “present for my mother” finding the gift list that says “Mom's birthday”, and “how to brew a hot drink with spices” finding the chai recipe.

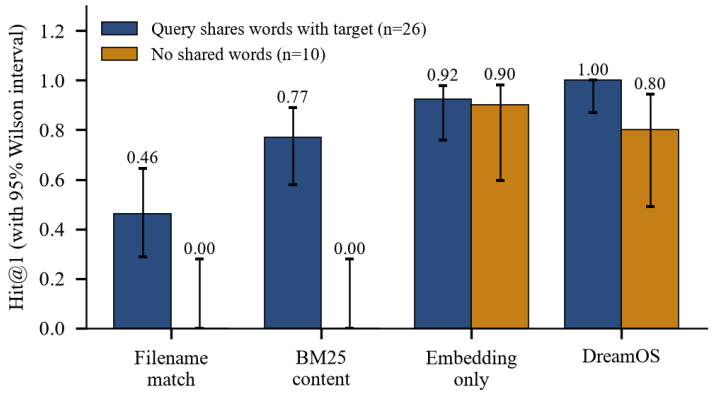


Figure 7.2 - Hit@1 by system and query group, with 95% Wilson intervals

Figure 7.2 shows Hit@1 for each system, split by whether the query shares a word with its target. The error bars are 95% Wilson score intervals, which are wide because the groups are small (26 and 10 queries). The lexical systems cannot answer queries that share no word with the target; the two embedding-based systems can.

Two results are less favourable and are reported as measured. First, the keyword boost helps queries that share words (Hit@1 0.92 to 1.00) but costs one of the 10 paraphrase queries (0.90 to 0.80); the intervals overlap, so the data cannot separate the two configurations. Second,

DreamOS missed rank 1 on two queries (“remarks from our mentor about what to improve” and “spending breakdown for the semester”), both of which placed an unrelated quotes file first with the correct file second. The baselines are reference implementations of the lexical class, not the Windows Search, Spotlight or Everything products, so the table supports a claim about lexical versus embedding retrieval, not about any named product.

7.3 Observations

Table 7.5 (page 28) summarises the effect of the defect fixes on the measured pipeline.

Table 7.5 - Effect of the defect fixes on the measured pipeline

Measure	Before fix	After fix	Change
Embedding call latency on the test machine	about 1,000 ms (name resolution)	about 15 ms	about 65 times faster
Indexing, 100-file test (files/s)	0.41	19.6	about 48 times faster
Indexing, 500 files (files/s, includes graph)	12.4	19.9	about 1.6 times faster
Graph rebuild at 2,000 files (s)	246.1	1.3	about 195 times faster
Graph rebuild at 5,000 files (s)	143.3 (vectorised similarity only)	3.2	about 45 times faster

Three findings from the work matter beyond the code itself. First, a delay of about one second on every model call was caused by the way the machine resolved the name localhost, which tries an IPv6 address first. The fix was to use the IPv4 loopback address directly. Without measurement this would have been blamed on the model. Second, the graph computation was quadratic in the number of files: at 2,000 files it took about fourteen times as long as at 500 files for four times the files. Vectorising the similarity step helped, but profiling the 5,000-file run showed that nearly all of the remaining time was a different all-pairs step, the reference check, and then the clustering rescan; each was replaced and checked against the original by a test. Without profiling, the first fix would have been declared enough. Third, a live rehearsal exposed a failure that the automated tests had not covered: a follow-up question about a new topic was answered with an old result, because the model judged the sentence to refer back to the conversation. The fix gives direct evidence precedence over the model's judgement, and a regression test was written that fails on the old behaviour.

Retrieval on short queries remains the weakest part. A one-word query can rank a file that merely contains the word above the file the user meant. The system reports such results with a confidence label instead of acting on them, which keeps the user in control.

CONCLUSION AND FUTURE SCOPE

8.1 Summary of Work

DreamOS was built as a local, meaning-based file manager with six modules, a desktop interface and a privacy audit. It indexes a folder with local embeddings, answers natural-language requests, opens files only on confident matches, organises files reversibly, relates files through a knowledge graph, remembers conversation so that follow-ups work, and recommends workspaces. In a controlled comparison it placed the intended file first for 94% of 36 labelled queries, against 56% for a BM25 content index, and answered paraphrased queries that lexical indexes cannot. It was measured on collections of up to 5,000 files, and the measurements led to fixes for the name-resolution delay, three all-pairs graph computations and one behavioural defect, each covered by a test.

8.2 Limitations

- The retrieval comparison uses 36 queries on a 37-file vault, written and labelled by the project team; the intervals are wide and the baselines are reference implementations, not the commercial products.
- The scale evaluation uses synthetic documents. Retrieval quality on real personal files has not been measured.
- Short, generic queries can rank a file that contains the word above the intended file.
- The 3-billion-parameter model misclassifies some borderline wording, for example routing “which files should I archive” to organise rather than workspace; intent accuracy was 88% on held-out phrasings.
- The similarity step builds a dense matrix of every pair of files, so its memory grows with the square of the collection (about 200 MB at 5,000 files). Collections beyond roughly ten thousand files would need an approximate nearest-neighbour index.
- The privacy audit checks configuration; it does not capture network traffic. The report describes the netstat check that does.
- The system runs on Windows only and handles text, Markdown, source, PDF and Word files.

8.3 Future Enhancement

- Evaluate retrieval on a labelled set of real documents, with the participants' consent and without storing their content.

- Run the same queries against the shipped Windows Search, Spotlight and Everything products for a head-to-head comparison.
- Replace the pairwise graph with an approximate nearest-neighbour index for collections above 10,000 files.
- Test the embedding task prefixes end to end, now that a labelled evaluation exists to judge them.
- Add an image and scanned-document pipeline with local optical character recognition.
- Package the model runtime with the installer so that first use does not require a separate download.

REFERENCES

[1] Z. Shi et al., “From Commands to Prompts: LLM-based Semantic File System for AIOS,” in Proc. ICLR 2025, arXiv:2410.11843.

[2] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019.

[4] Chroma, “Chroma: the open-source embedding database,” github.com/chroma-core/chroma.

[5] Microsoft, “Windows Search overview,” Microsoft Learn documentation.

[6] Apple Inc., “Spotlight User Guide,” support.apple.com.

[7] voidtools, “Everything - locate files and folders by name instantly,” voidtools.com.

[8] Ollama, “Ollama: run large language models locally,” ollama.com.

[9] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, 2009.

[10] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” J. Amer. Statist. Assoc., vol. 22, no. 158, 1927.

APPENDICES

Appendix A Source code

The complete source code is in the private repository github.com/zaidwhy/dreamos-college-project, which is available to the examiners on request. The main files are backend/app (service), frontend/src (interface), backend/tests (automated tests), backend/scripts/bench_scale.py (scale benchmark) and backend/scripts/eval_retrieval.py (retrieval comparison).

Appendix B Scale benchmark raw results

Table B.1 (page 33) gives the raw benchmark output at each size.

Table B.1 - Raw benchmark output at 500, 2,000 and 5,000 files

Measure	500 files	2,000 files	5,000 files
Files indexed	500	2000	5000
Index time (s)	25.2	86.1	276.3
Graph rebuild (s)	0.4	1.3	3.2
Graph links	1062	4747	11938
Search p50 / p95 (ms)	34.5 / 60.7	106.5 / 126.7	204.7 / 232.7

Appendix C Research paper

A research paper on this project, prepared in IEEE conference format for the Research Methodology CA4 requirement, is included with this report (docs/paper/DreamOS-IEEE-Paper.pdf). Its figures, tables and measurements are the same as those in this report.

Appendix D Papers and certificates

No research paper has been accepted or presented as part of this project at the time of writing. Acceptance letters and presentation certificates, if any are issued, will be attached here.